IS107 Laboratory Work 5

Applying DAX Fundamentals in Power BI

From Measures to Contextual Analysis

Submitted by: Dianah Myra Salazar| Power BI Desktop

Data Model Overview

Customers

- Customer ID
- Name
- Address / State
- Phone, Zip
- Notes

Orders

- Order ID / Customer ID
- Product, Units Sold
- Date, Revenue, Cost
- Profit, Day of week
- Has Chocolate

Cookie Types

- Cookie Type
- Units Sold
- Revenue Per Cookie
- Cost Per Cookie

Cookie Types → Orders (1:) · Customers → Orders (1:*)*

I. Foundational Concepts — Q1 & Q2

Q1 — Excel Formula vs. DAX

For me, **Excel formulas** are static — they compute one cell at a time and don't travel across related tables. **DAX** is a formula language built for **relational data models**. My thought is that DAX understands table relationships, automatically aggregates across millions of rows, and re-evaluates the moment a filter or slicer changes — something a plain Excel SUM formula simply cannot do. While Excel calculates a cell value, DAX calculates a **context-aware result**.

Q2 — The Why of DAX

Raw numbers alone can't answer questions like *"What is my profit margin per cookie type for orders above 500 units?"* In my view, the gap DAX fills is **dynamic aggregation with context**. The data in our **Orders** table is flat — it has Revenue and Cost per row, but no formula tells Power BI **how to sum, filter, or relate** them to Cookie Types or Customers. DAX bridges that gap by letting us define reusable logic (Measures) that respond to every visual and filter on the canvas.

II. Calculated Columns vs. Measures — Q3 & Q4

	Calculated Column	Measure
Storage	Stored in the model — increases .pbix file size	Computed on the fly — no extra storage
Evaluation	Row-by-row at refresh time	Evaluated per visual / filter context
Best for	Row-level flags (e.g. Has Chocolate)	Aggregations (Sum, Count, %)
Example	Day of week, Has Chocolate column	Total Profit, Count of orders, Profit Margin %

Q4 — How Filter Context Affects a Measure

My thoughts: In my dashboard, when no filter is active, "Count of orders" shows 700 (total). The moment I click a customer row — say ACME Bites — the filter context narrows the evaluation space and the measure instantly returns 206. DAX's CALCULATE function is at the heart of this: it re-evaluates the measure expression inside a modified filter context, making every card, chart, and table automatically reactive to slicers and cross-highlights.

III. Function Application & Syntax — Q5 & Q6

Q5 — The RELATED Function

For me, RELATED is the bridge between tables. When I was building the ***Has Chocolate*** calculated column inside **Orders**, I needed to look up whether the product belonged to a chocolate category in **Cookie Types**. RELATED follows the established relationship to fetch a value from the related table — but it only works when a **one-to-many relationship** has been defined in the **Model View**. Without that relationship, DAX has no path to traverse and RELATED returns an error. In our model, Cookie Types (1) → Orders (*) via Product enables this lookup.

Q6 — Deconstructing the Formula

```
Total Sales = SUM(Sales[Sales Amount])
```

Measure Name:	Total Sales — descriptive, unique identifier for the KPI
Function:	SUM( ) — DAX aggregation function; iterates every row in scope
Table:	Sales — explicitly scopes the column lookup to avoid ambiguity
Column:	[Sales Amount] — the specific field being aggregated
Best Practice:	Always prefix with the table name (Table[Column]) so DAX can resolve columns unambiguously across multiple tables — especially critical in our 3-table model.

Visual Results — Key Outputs

Total Units Sold by Customer

ABC Groceries: 221,023

ACME Bites: 329,677

Park & Shop: 179,448

Tres Delicious: 128,769

Wholesome: 266,890

TOTAL: 1,125,806

Count of Orders by Customer

ABC Groceries: 132

ACME Bites: 206

Park & Shop: 114

Tres Delicious: 92

Wholesome Foods: 156

TOTAL: 700

Profit Margin % by Cookie

Choc Chip: 60.00%

Fortune Cookie: 50.00%

Oatmeal Raisin: 56.00%

Snickerdoodle: 62.50%

Sugar: 58.33%

TOTAL: 57.93%

Total Profit (Card)

Total Profit 2:

2,717,064.38

Revenue – Cost = Profit

Across all customers

& all cookie types

Choc Chip Orders > 500 units

Count: 185

DAX CALCULATE used

with FILTER condition

Units Sold > 500

on Chocolate Chip rows

Distinct Customers / Profit

Each customer: 1

unique buyer (5 total)

Total Profit: 2,717,070.50

Revenue – Cost

confirmed equal

across all rows

IV. Analysis & Troubleshooting — Q7 & Q8

Q7 — Negative Total Profit: Broken Formula or Business Insight?

My thoughts: A negative Total Profit figure does NOT mean the DAX formula is broken. For me, it means the formula is working exactly as designed — it's telling me that for that specific category, costs exceeded revenue. The data is the problem, not the logic. To verify: (1) Cross-check the Revenue and Cost columns in the raw Orders table for that category. (2) Confirm the Profit calculated column formula is correct: Profit = Revenue – Cost. (3) Check if there are data-entry errors (e.g. reversed Revenue/Cost values). In our actual results, all profits were positive (Total Profit 2 = 2,717,064.38), which confirms the formula is healthy and the business is profitable across all segments.

Q8 — Profit Margin %: Calculated Column vs. Measure — Which is More Efficient?

A single Measure is far more efficient. For me, creating a Calculated Column for Profit Margin % would compute and store a value for every one of the 700 rows in our Orders table — bloating file size and recalculating at every model refresh. My thought is that a Measure defined as:

```
Profit Margin % = DIVIDE(SUM(Orders[Profit]), SUM(Orders[Revenue]))
```

...computes nothing until a visual requests it, and then it computes only the aggregated result scoped to the current filter context. This is both memory-efficient and dynamically correct — if I slice by Cookie Type, the measure automatically re-evaluates for just that subset, while a calculated column would require rebuilding entirely.

Key Takeaways

- DAX operates on relational models with context-awareness — not just static cell math.
- Calculated Columns store row-level data and grow the file; Measures are on-demand and storage-lean.
- Filter Context is what makes a Measure dynamic — the same formula yields different results per visual.
- RELATED() requires established model relationships to bridge tables in column formulas.
- Profit Margin % is best built as a Measure for efficiency and dynamic filtering.
- Unexpected results (negatives) indicate business data issues — not broken DAX logic.

Appendix

Visual Outputs & Screenshots

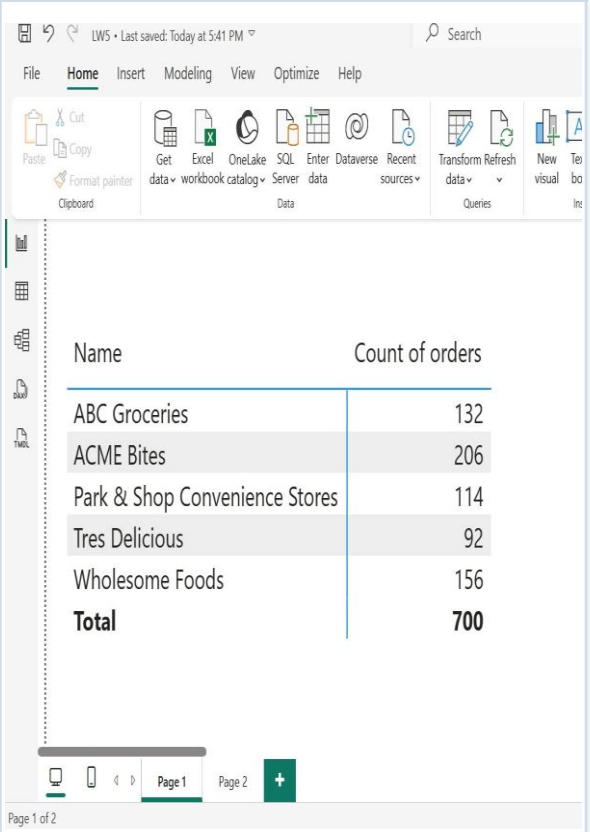
A: Power BI Dashboard Visuals | **B:** Data Tables | **C:** Model Relationship Diagram

Appendix A — Dashboard Visuals (1/2)

Total Number of Units Sold by Customer

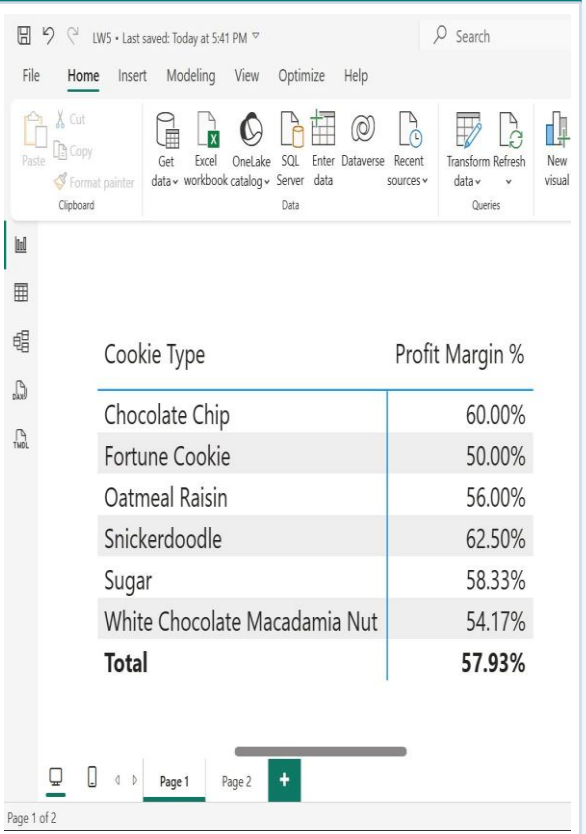
Name	Total Number of Units sold
ABC Groceries	221,023
ACME Bites	329,677
Park & Shop Convenience Stores	179,448
Tres Delicious	128,769
Wholesome Foods	266,890
Total	**1,125,806**

Count of Orders by Customer



Name	Count of orders
ABC Groceries	132
ACME Bites	206
Park & Shop Convenience Stores	114
Tres Delicious	92
Wholesome Foods	156
Total	**700**

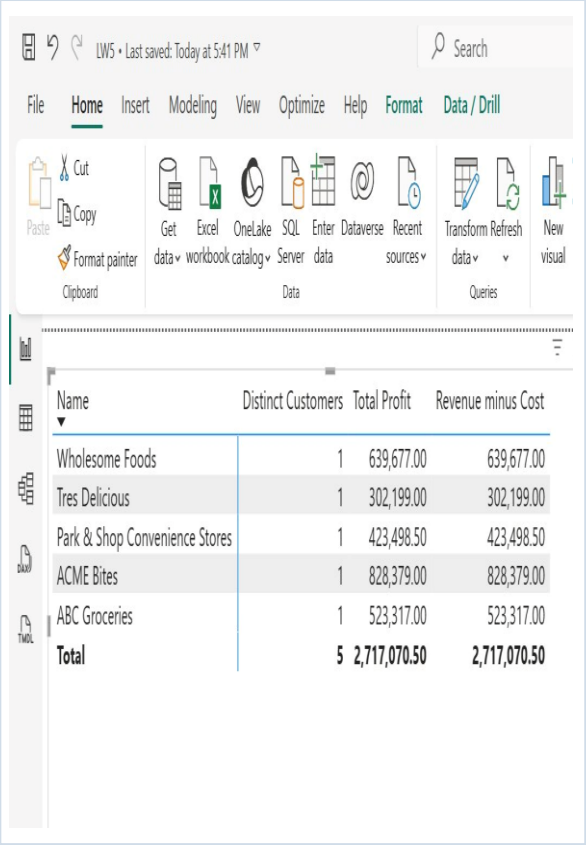
Profit Margin % by Cookie Type



Cookie Type	Profit Margin %
Chocolate Chip	60.00%
Fortune Cookie	50.00%
Oatmeal Raisin	56.00%
Snickerdoodle	62.50%
Sugar	58.33%
White Chocolate Macadamia Nut	54.17%
Total	**57.93%**

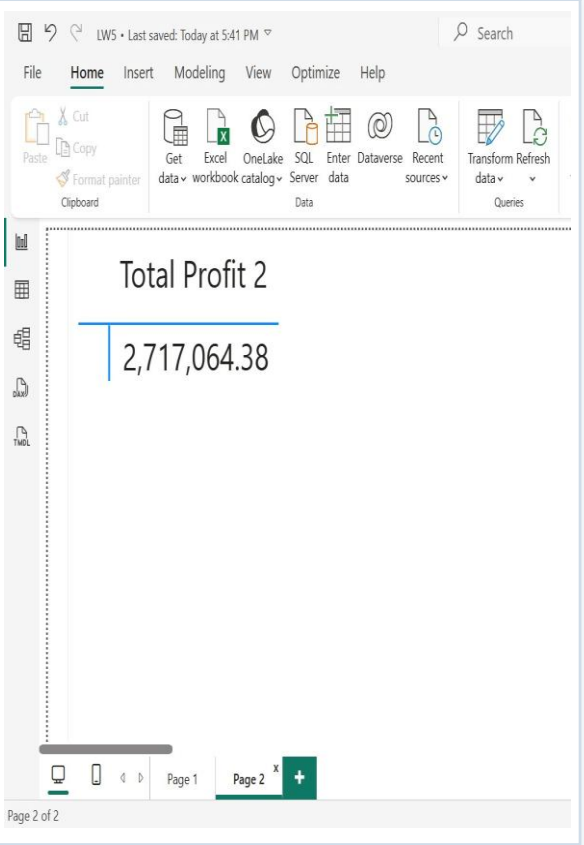
Appendix A — Dashboard Visuals (2/2)

Distinct Customers, Total Profit & Revenue minus Cost

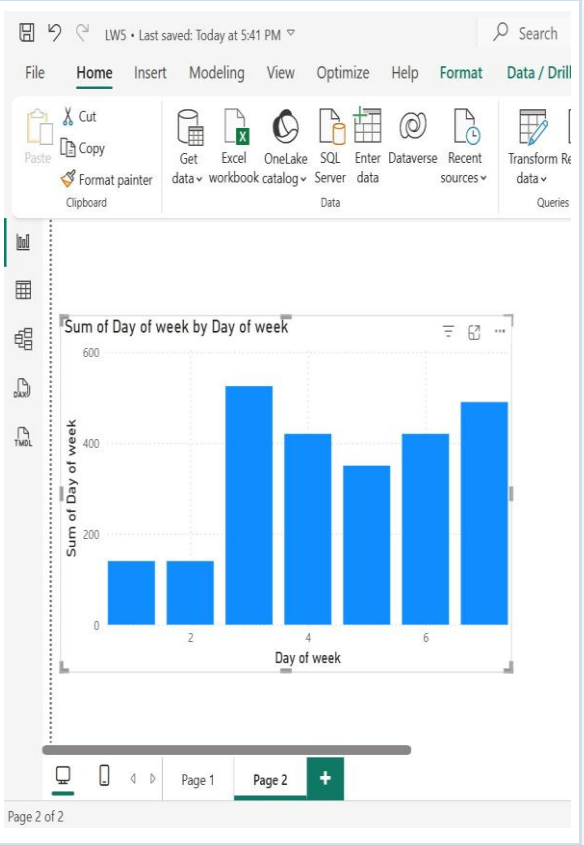


Name	Distinct Customers	Total Profit	Revenue minus Cost
Wholesome Foods	1	639,677.00	639,677.00
Tres Delicious	1	302,199.00	302,199.00
Park & Shop Convenience Stores	1	423,498.50	423,498.50
ACME Bites	1	828,379.00	828,379.00
ABC Groceries	1	523,317.00	523,317.00
Total	**5**	**2,717,070.50**	**2,717,070.50**

Total Profit 2 (Card Visual)

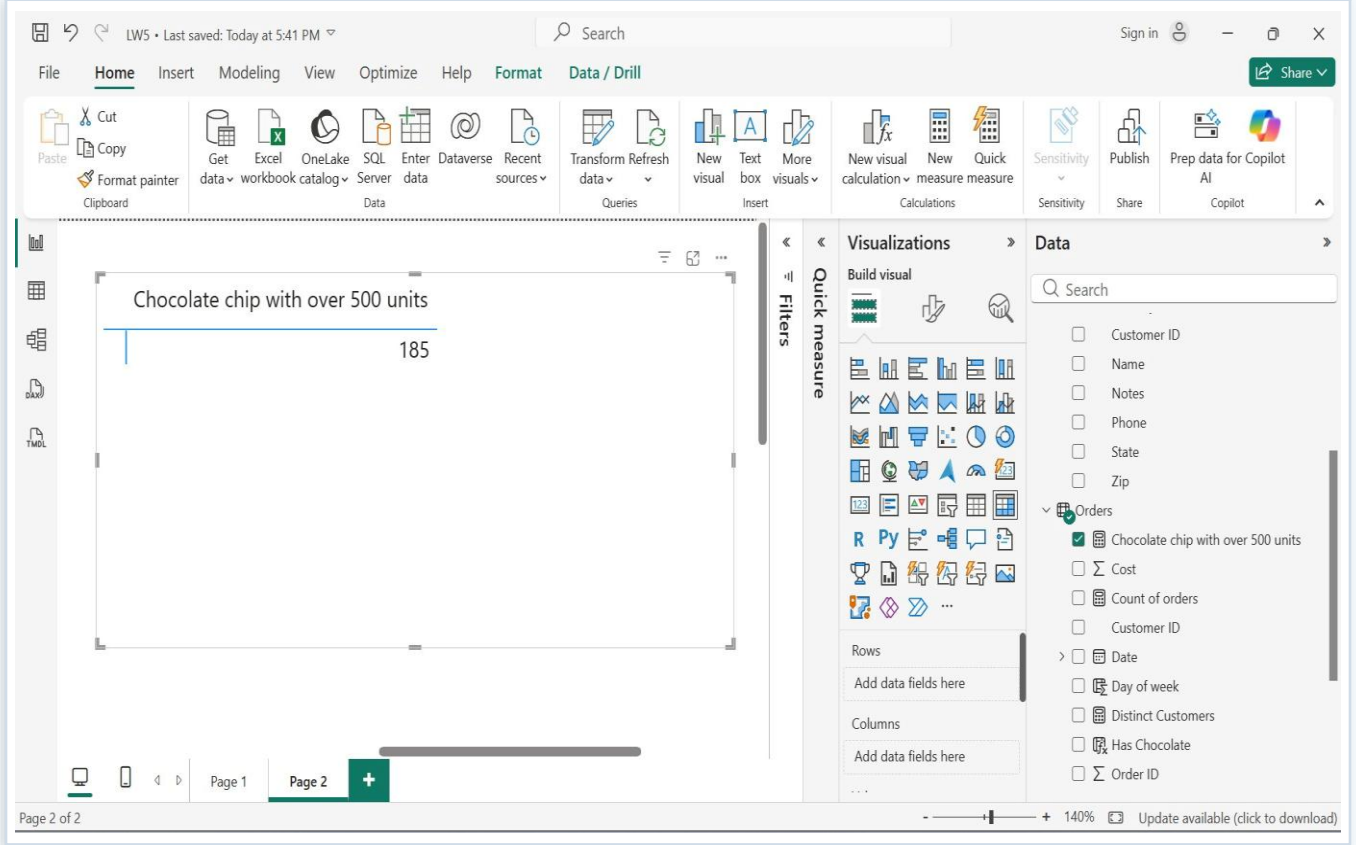


Sum of Day of Week by Day of Week (Bar Chart)



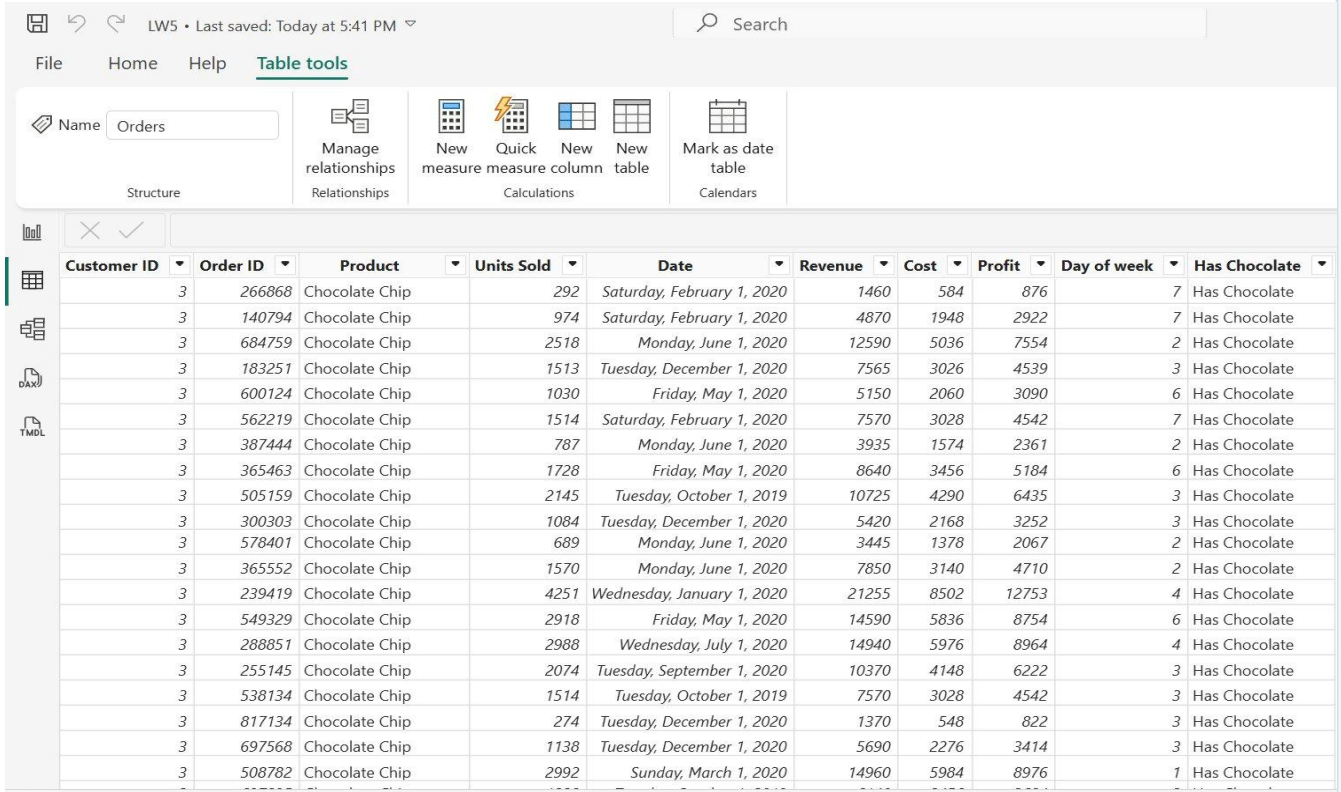
Appendix A — Dashboard Visuals (3/3)

Chocolate Chip Orders with over 500 Units



Appendix B — Data Tables

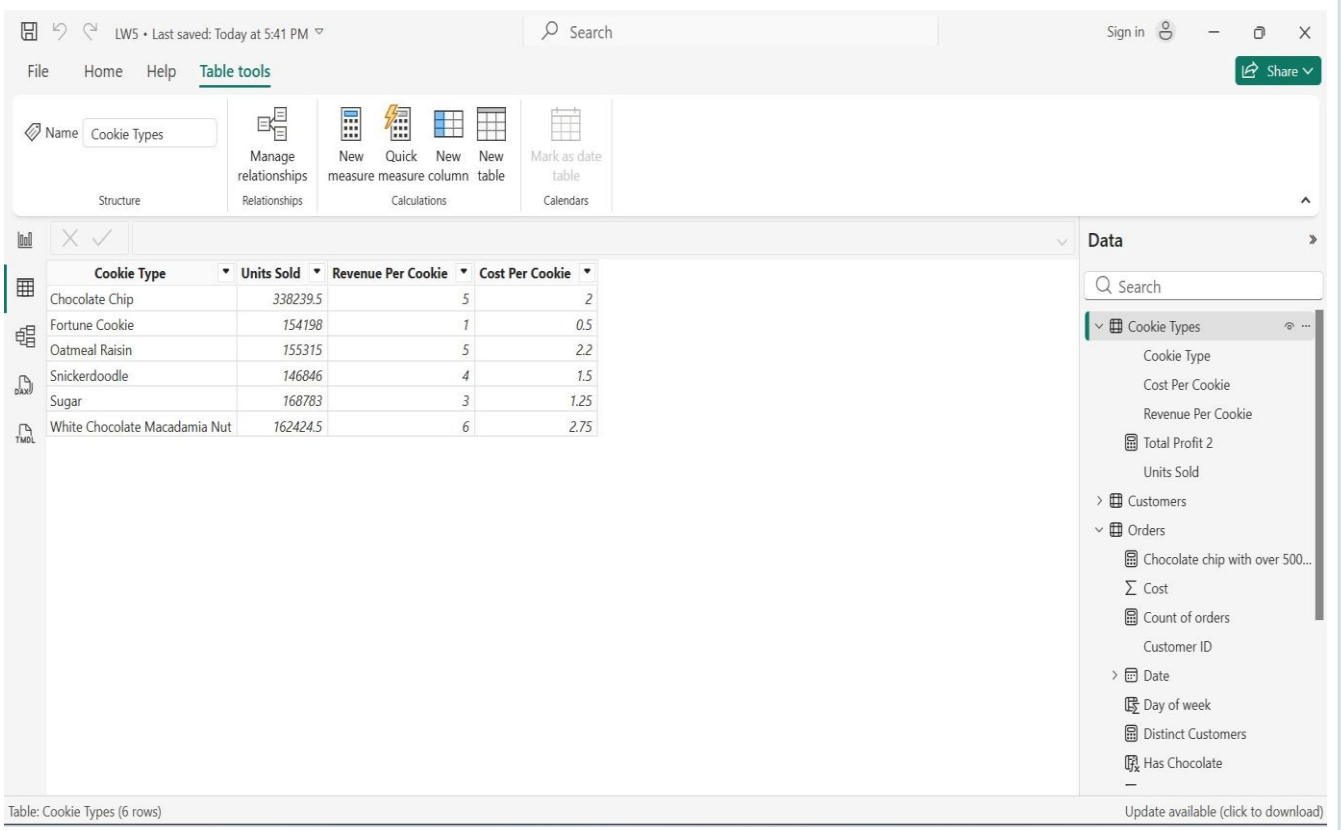
Orders Table — Raw Data View (700 rows)



Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit	Day of week	Has Chocolate
3	266868	Chocolate Chip	292	Saturday, February 1, 2020	1460	584	876	7	Has Chocolate
3	140794	Chocolate Chip	974	Saturday, February 1, 2020	4870	1948	2922	7	Has Chocolate
3	684759	Chocolate Chip	2518	Monday, June 1, 2020	12590	5036	7554	2	Has Chocolate
3	183251	Chocolate Chip	1513	Tuesday, December 1, 2020	7565	3026	4539	3	Has Chocolate
3	600124	Chocolate Chip	1030	Friday, May 1, 2020	5150	2060	3090	6	Has Chocolate
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542	7	Has Chocolate
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361	2	Has Chocolate
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184	6	Has Chocolate
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435	3	Has Chocolate
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252	3	Has Chocolate
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067	2	Has Chocolate
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710	2	Has Chocolate
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753	4	Has Chocolate
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754	6	Has Chocolate
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964	4	Has Chocolate
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222	3	Has Chocolate
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542	3	Has Chocolate
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822	3	Has Chocolate
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414	3	Has Chocolate
3	508782	Chocolate Chip	2992	Sunday, March 1, 2020	14960	5984	8976	1	Has Chocolate

Appendix B — Data Tables

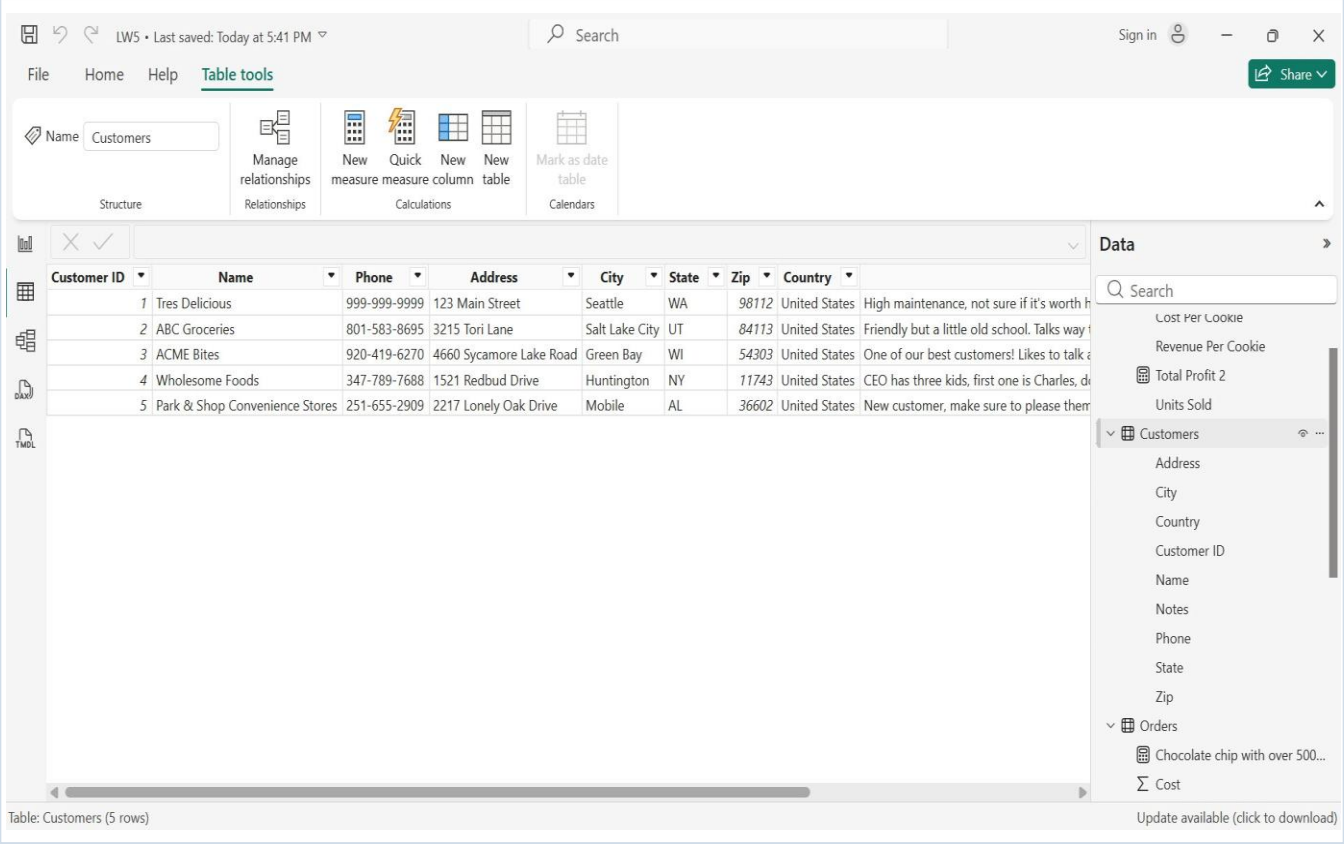
Cookie Types Table — Data View (6 rows)



Cookie Type	Units Sold	Revenue Per Cookie	Cost Per Cookie
Chocolate Chip	338239.5	5	2
Fortune Cookie	154198	1	0.5
Oatmeal Raisin	155315	5	2.2
Snickerdoodle	146846	4	1.5
Sugar	168783	3	1.25
White Chocolate Macadamia Nut	162424.5	6	2.75

Appendix B — Data Tables

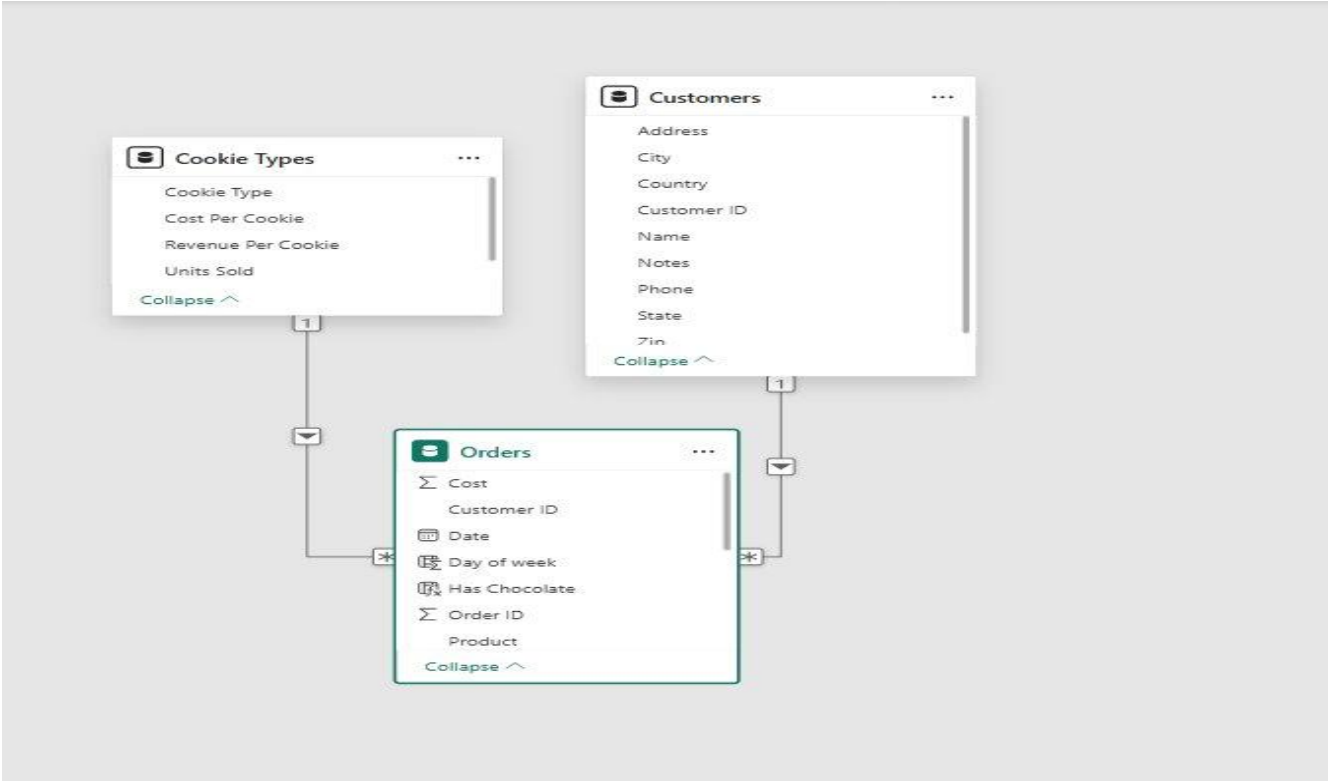
Customers Table — Data View (5 rows)



Customer ID	Name	Phone	Address	City	State	Zip	Country	
1	Tres Delicious	999-999-9999	123 Main Street	Seattle	WA	98112	United States	High maintenance, not sure if it's worth h
2	ABC Groceries	801-583-8695	3215 Tori Lane	Salt Lake City	UT	84113	United States	Friendly but a little old school. Talks way t
3	ACME Bites	920-419-6270	4660 Sycamore Lake Road	Green Bay	WI	54303	United States	One of our best customers! Likes to talk a
4	Wholesome Foods	347-789-7688	1521 Redbud Drive	Huntington	NY	11743	United States	CEO has three kids, first one is Charles, d
5	Park & Shop Convenience Stores	251-655-2909	2217 Lonely Oak Drive	Mobile	AL	36602	United States	New customer, make sure to please them

Appendix C — Model Relationship Diagram

Model View — Table Relationships Diagram



The model establishes two 1-to-many relationships: Cookie Types → Orders (via Product) and Customers → Orders (via Customer ID). These relationships enable RELATED() lookups and cross-table DAX measures.

DAX Measure Formulas

Calculated Columns & Measures

2 Calculated Columns	*Profit · Day of week*
1 IF Column	*Has Chocolate*
1 CALCULATE Measure	*Choc Chip > 500 units*
1 Quick Measure	*Revenue minus Cost*

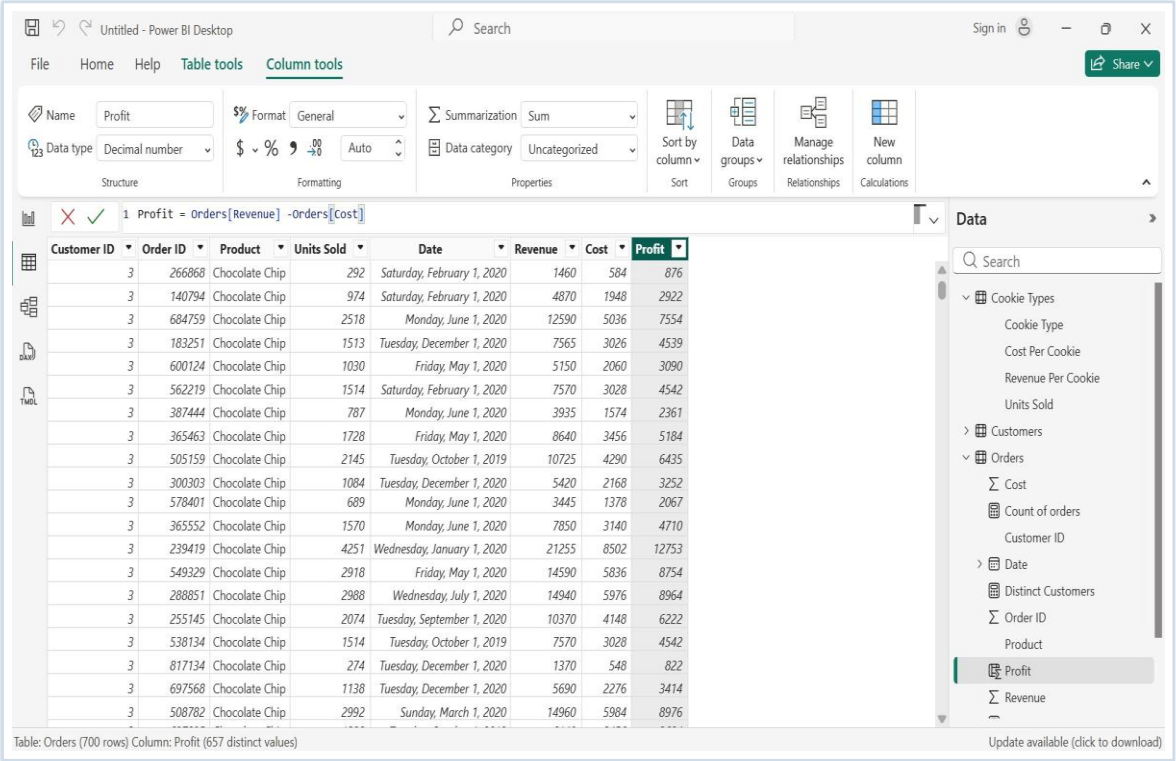
fx

```
Profit
  = Orders[Revenue]
  - Orders[Cost]

Day of week
  = WEEKDAY(Orders[Date],1)

Has Chocolate
  = IF(FIND(...) > 0,
    "Has Chocolate",
    "No Chocolate")

Chocolate chip > 500
  = CALCULATE(COUNTROWS(Orders),
    Orders[Units Sold] > 500,
    Orders[Product]=="Chocolate Chip")
```

Calculated Column 1 — Profit

Orders Table

```
fx  Profit  =  Orders[Revenue]  -  Orders[Cost]
```



Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit
3	266868	Chocolate Chip	292	Saturday, February 1, 2020	1460	584	876
3	140794	Chocolate Chip	974	Saturday, February 1, 2020	4870	1948	2922
3	684759	Chocolate Chip	2518	Monday, June 1, 2020	12590	5036	7554
3	183251	Chocolate Chip	1513	Tuesday, December 1, 2020	7565	3026	4539
3	600124	Chocolate Chip	1030	Friday, May 1, 2020	5150	2060	3090
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414
3	508782	Chocolate Chip	2992	Sunday, March 1, 2020	14960	5984	8976

About this Formula

Type

Calculated Column

Table

Orders

Output

Decimal number (Profit per row)

Logic

Subtracts Cost from Revenue for every single row in the Orders table at data refresh time

Why Column?

Each order has a unique Revenue & Cost — a row-level calculation is needed, not an aggregate

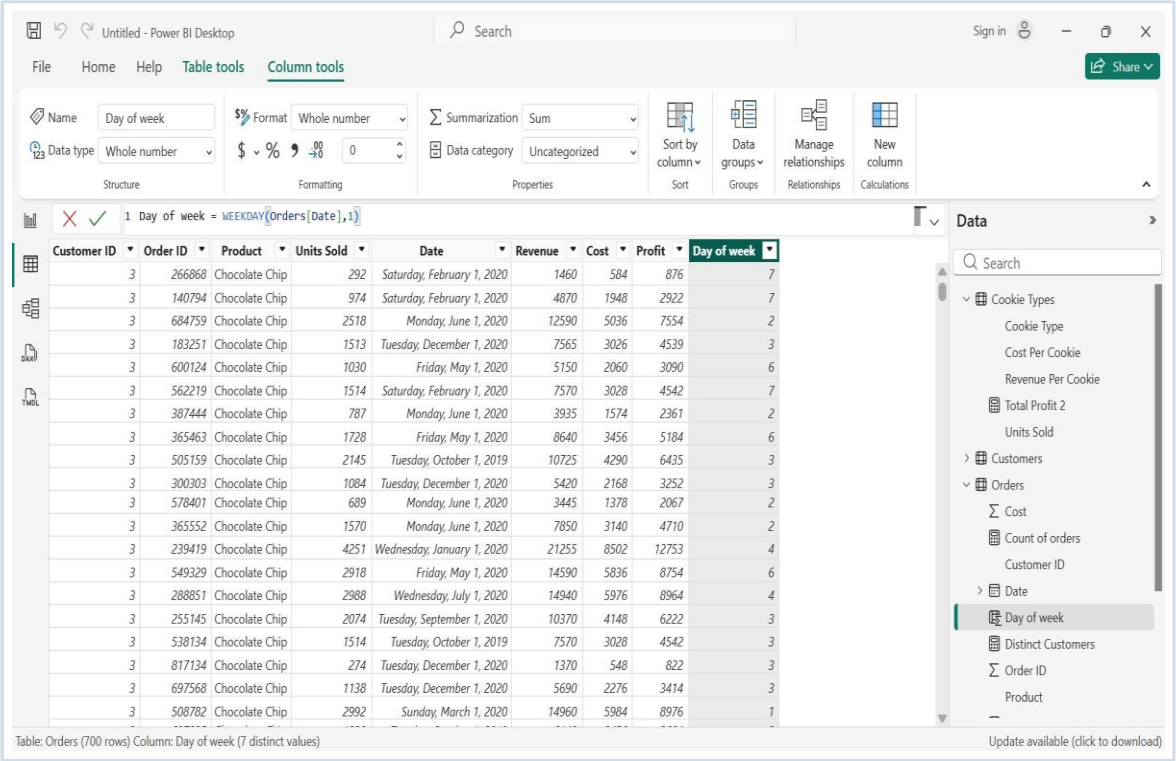
Result

657 distinct profit values across 700 rows

Calculated Column 2 — Day of Week

Orders Table

```
fx  Day of week  =  WEEKDAY(  Orders[Date]  ,1)
```



Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit	Day of week
3	266868	Chocolate Chip	292	Saturday, February 1, 2020	1460	584	876	7
3	140794	Chocolate Chip	974	Saturday, February 1, 2020	4870	1948	2922	7
3	684759	Chocolate Chip	2518	Monday, June 1, 2020	12590	5036	7554	2
3	183251	Chocolate Chip	1513	Tuesday, December 1, 2020	7565	3026	4539	3
3	600124	Chocolate Chip	1030	Friday, May 1, 2020	5150	2060	3090	6
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542	7
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361	2
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184	6
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435	3
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252	3
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067	2
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710	2
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753	4
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754	6
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964	4
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222	3
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542	3
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822	3
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414	3
3	508782	Chocolate Chip	2992	Sunday, March 1, 2020	14960	5984	8976	1

About this Formula

Type

Calculated Column

Table

Orders

Function

WEEKDAY( ) — DAX date function

Param ,1

Return type 1 = Sunday(1) to Saturday(7)

Output

Whole number (1–7) per row

Use Case

Enables grouping and charting orders by day of the week in the bar chart visual

Result

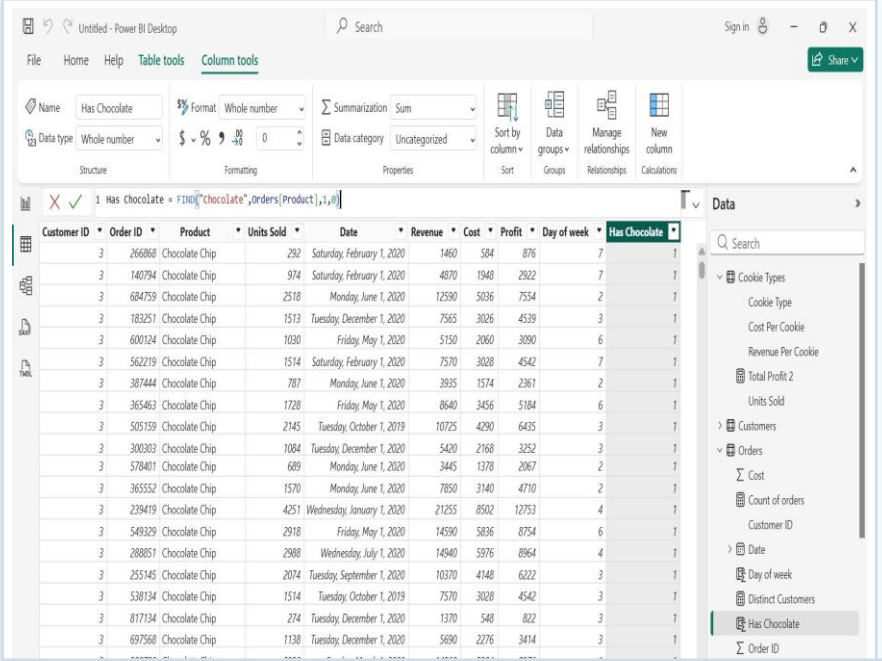
7 distinct day values across all 700 rows

Calculated Column 3 — Has Chocolate

Orders Table

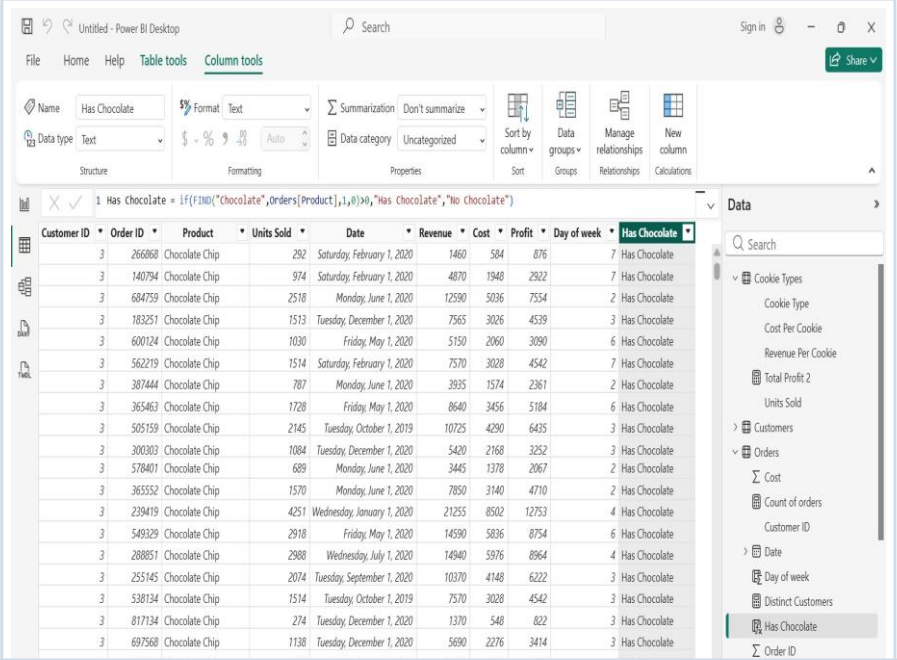
```
fx  Has Chocolate        = IF(FIND(    "Chocolate"     ,Orders[Product],1,0)>0,
    "Has Chocolate","No Chocolate")
```

Step 1 — Initial FIND formula



Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit	Day of week	Has Chocolate
3	266868	Chocolate Chip	292	Saturday, February 1, 2020	1460	584	876	7	1
3	140794	Chocolate Chip	974	Saturday, February 1, 2020	4870	1948	2922	7	1
3	684759	Chocolate Chip	2518	Monday, June 1, 2020	12590	5036	7554	2	1
3	183251	Chocolate Chip	1513	Tuesday, December 1, 2020	7565	3026	4539	3	1
3	600124	Chocolate Chip	1030	Friday, May 1, 2020	5150	2060	3090	6	1
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542	7	1
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361	2	1
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184	6	1
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435	3	1
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252	3	1
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067	2	1
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710	2	1
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753	4	1
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754	6	1
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964	4	1
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222	3	1
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542	3	1
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822	3	1
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414	3	1

Step 2 — Final IF + FIND formula (text output)



Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit	Day of week	Has Chocolate
3	266868	Chocolate Chip	292	Saturday, February 1, 2020	1460	584	876	7	Has Chocolate
3	140794	Chocolate Chip	974	Saturday, February 1, 2020	4870	1948	2922	7	Has Chocolate
3	684759	Chocolate Chip	2518	Monday, June 1, 2020	12590	5036	7554	2	Has Chocolate
3	183251	Chocolate Chip	1513	Tuesday, December 1, 2020	7565	3026	4539	3	Has Chocolate
3	600124	Chocolate Chip	1030	Friday, May 1, 2020	5150	2060	3090	6	Has Chocolate
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542	7	Has Chocolate
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361	2	Has Chocolate
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184	6	Has Chocolate
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435	3	Has Chocolate
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252	3	Has Chocolate
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067	2	Has Chocolate
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710	2	Has Chocolate
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753	4	Has Chocolate
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754	6	Has Chocolate
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964	4	Has Chocolate
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222	3	Has Chocolate
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542	3	Has Chocolate
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822	3	Has Chocolate
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414	3	Has Chocolate

FIND( ) searches for 'Chocolate' in the Product column. IF( ) wraps it to return a readable label — 'Has Chocolate' or 'No Chocolate' — making the column usable as a categorical filter.

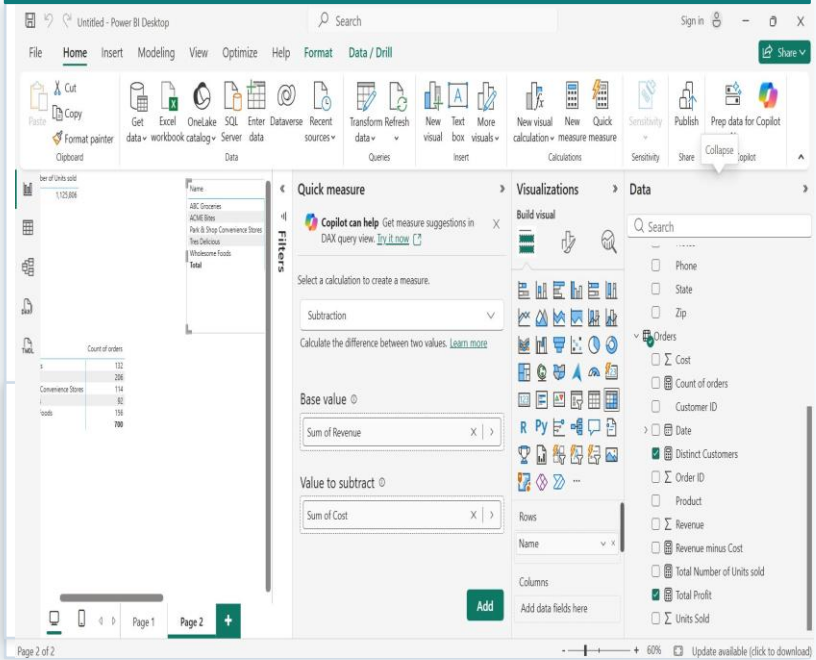
Quick Measure — Revenue minus Cost

Orders Table · Subtraction

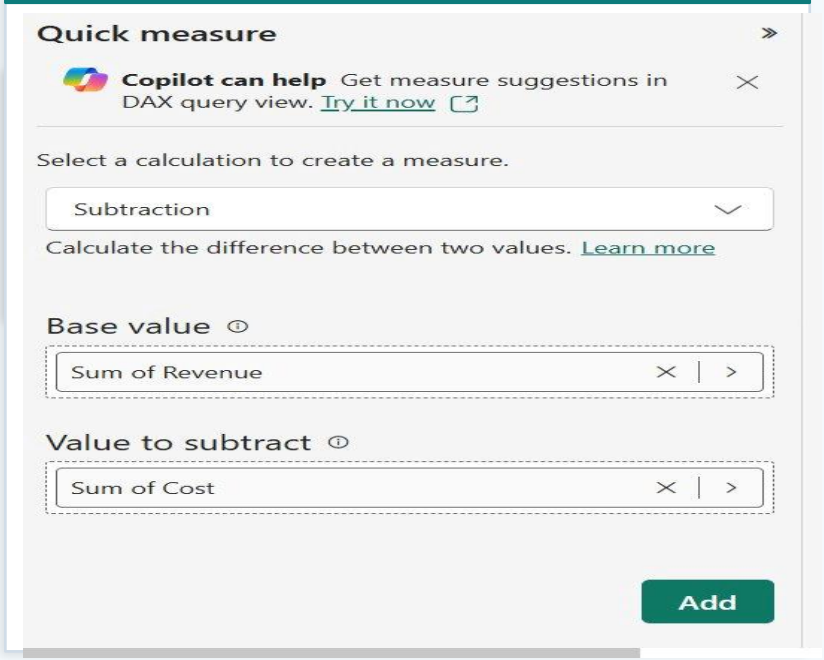
Quick Measure — auto-generated DAX:

```
Revenue minus Cost = SUM(Orders[Revenue]) - SUM(Orders[Cost])
```

Measure added — visible in Data panel



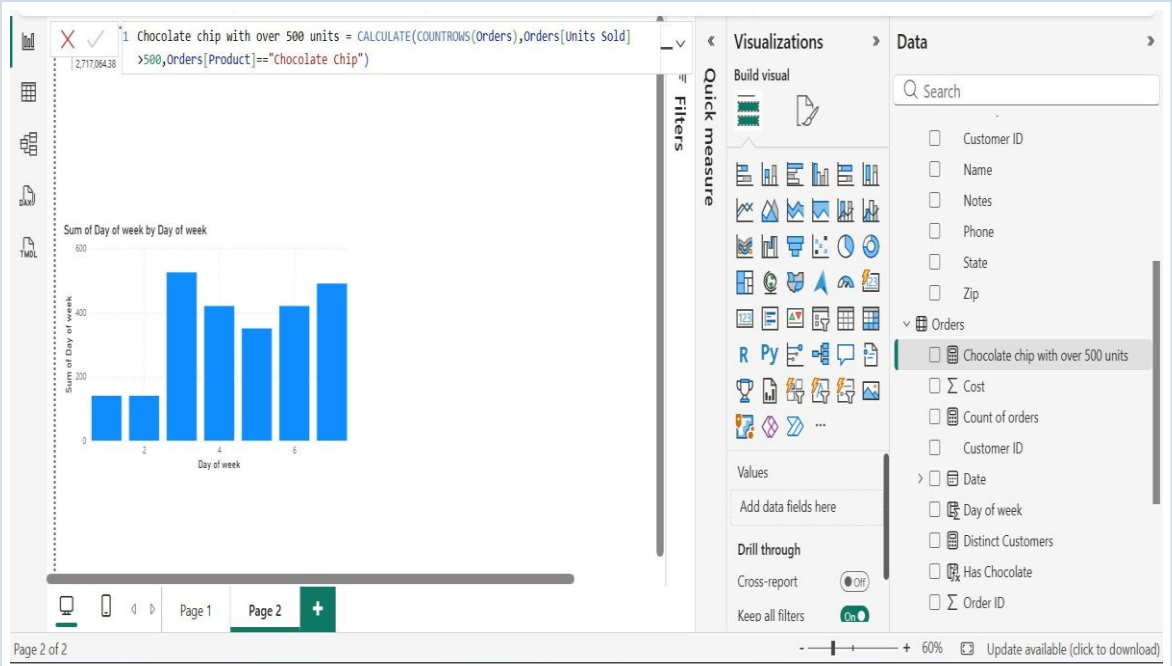
Quick Measure panel — Subtraction selected



Power BI's Quick Measure wizard auto-generates this DAX using Subtraction: Base Value (Sum of Revenue) − Value to Subtract (Sum of Cost). Unlike the Profit column, this is a Measure — it aggregates dynamically based on filter context.

CALCULATE Measure — Chocolate Chip with over 500 Units

```
fx  Chocolate chip with over 500 units
      = CALCULATE(          COUNTROWS(Orders)
    ,  Orders[Units Sold] > 500              ,  Orders[Product] ==        "Chocolate Chip")
```



About this Formula

Type

Measure (DAX CALCULATE)

Function

CALCULATE( ) — modifies filter context before evaluating

Filter 1

Orders[Units Sold] > 500 — only large orders

Filter 2

Orders[Product] == "Chocolate Chip" — product filter

Returns

Count of rows matching BOTH filters

Result

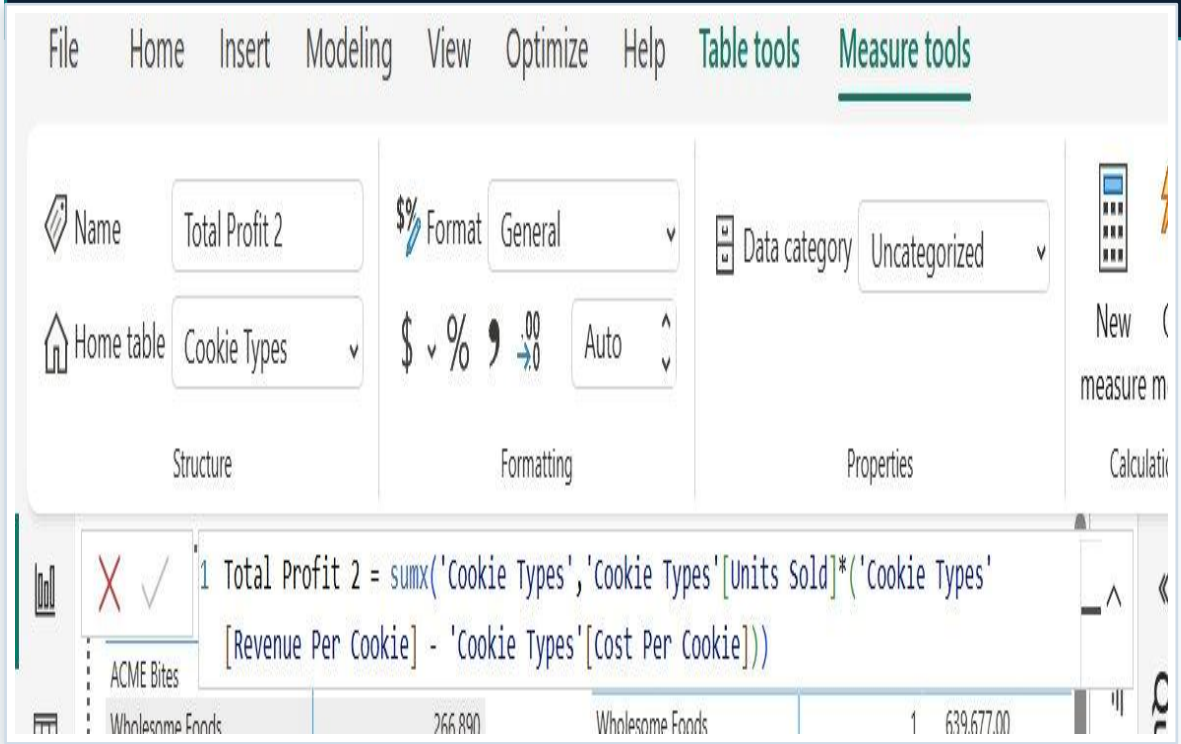
185 orders matched

Why Measure?

No storage needed; re-evaluates dynamically if slicers change

Measure — Total Profit 2


```
Total Profit 2 = SUMX( 'Cookie Types' , 'Cookie Types' [Units Sold]
  * ( 'Cookie Types'[Revenue Per Cookie] - 'Cookie Types'[Cost Per Cookie] ) )
```



About this Formula

Type

Measure (SUMX iterator)

Table

Cookie Types

Function

SUMX( ) — iterates each row, evaluates expression, sums results

Logic

For each cookie type: Units Sold × (Revenue Per Cookie − Cost Per Cookie)

Why SUMX?

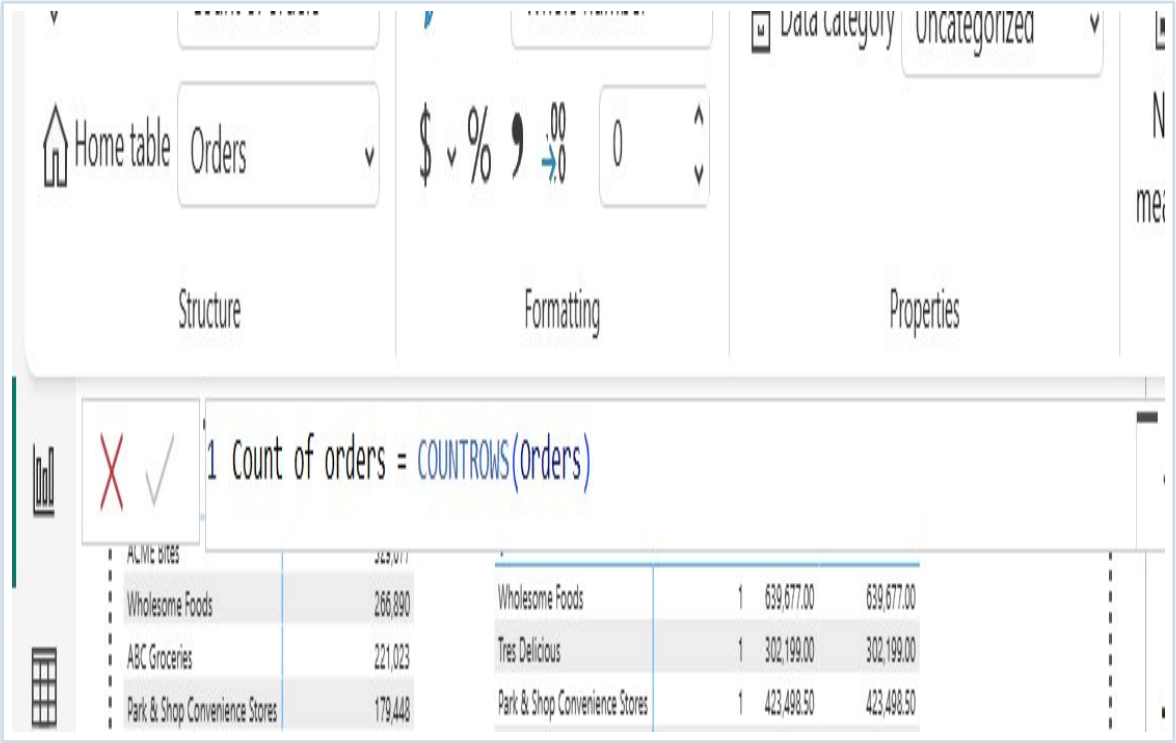
Row-level math needed before aggregating — SUM alone can't multiply then sum

Result

Total profit across all cookie types

Measure — Count of Orders


```
fx  Count of orders  =  COUNTROWS(  Orders  )
```



About this Formula

Use Case

Drives the Count of Orders table visual on the dashboard

Table

Orders

Function

COUNTROWS( ) — counts all rows in the Orders table

Context

Responds to filter context — clicking a customer narrows the count to their orders only

No filter

Returns 700 (all orders)

Filtered

e.g. ACME Bites → 206 orders

Measure — Distinct Customers

Orders Table

```
fx  Distinct Customers  =  DISTINCTCOUNT(  Orders[Customer ID]  )
```

```
// This tells me how many customers we have  ← DAX comment line
```

About this Formula

Use Case

Confirms the number of unique buyers in the Distinct Customers visual

Table

Orders

Type

Measure

Function

DISTINCTCOUNT( ) — counts unique non-blank values in a column

Column

Orders[Customer ID] — the unique customer identifier

Comment

DAX supports // comments — used here to annotate intent

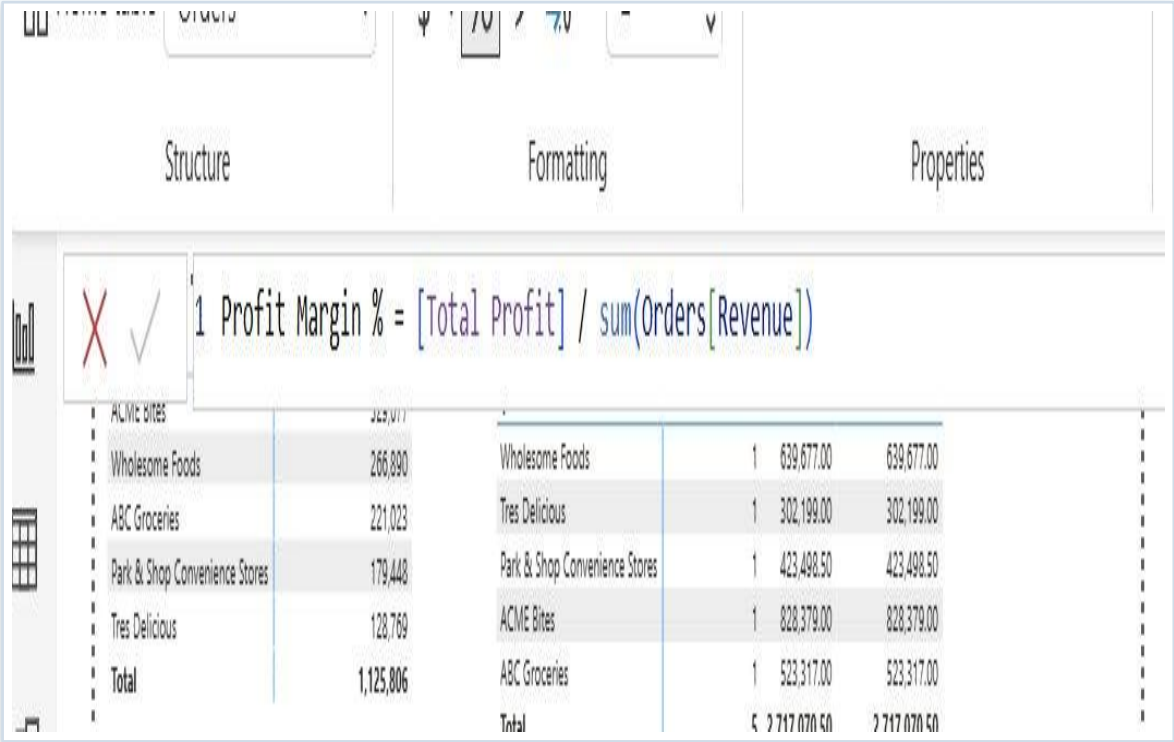
Result

5 distinct customers in total

Measure — Profit Margin %

Orders Table

```
fx  Profit Margin %  =  [Total Profit]  /  sum(  Orders[Revenue]  )
```



About this Formula

Best Practice

Measure references measure — keeps logic modular and reusable

Table

Orders

Type

Measure

Numerator

[Total Profit] — references another measure

Denominator

SUM(Orders[Revenue]) — total revenue in context

Output

Decimal ratio → formatted as % in visual (e.g. 57.93%)

Context

Filter context changes both numerator & denominator — Choc Chip shows 60.00%

Measure — Total Number of Units Sold


```
fx  Total Number of Units sold  =  SUM(  Orders[Units Sold]  )
```

Structure | Formatting | Properties

```
1 Total Number of Units sold = SUM(Orders[Units Sold])
```

ACME Bites	329,677
Wholesome Foods	266,890
ABC Groceries	221,023
Park & Shop Convenience Stores	179,448
Tres Delicious	128,769
Total	**1,125,806**

Wholesome Foods	1	639,677.00	639,677.00
Tres Delicious	1	302,199.00	302,199.00
Park & Shop Convenience Stores	1	423,498.50	423,498.50
ACME Bites	1	828,379.00	828,379.00
ABC Groceries	1	523,317.00	523,317.00
Total	**5**	**2,717,070.50**	**2,717,070.50**

About this Formula

Use Case

Drives the Total Number of Units Sold table visual

Table

Orders

Type

Measure

Function

SUM( ) — aggregates all values in Orders[Units Sold]

Result

1,125,806 total units across all 700 orders

By Customer

ACME Bites: 329,677 · Wholesome: 266,890 · etc.

Context

Slicing by customer or product automatically re-totals

Additional DAX Measures — Summary

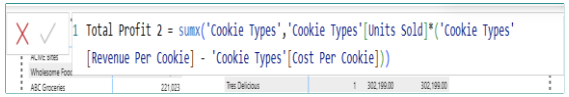
Measure (SUMX)

Total Profit 2

Total profit from Cookie Types table

```
Total Profit 2 = SUMX('Cookie Types','Cookie Types'[Units Sold]*('Cookie Types'[Revenue Per Cookie]-'Cookie Types'[Cost Per Cookie]))
```

Iterates Cookie Types table. Multiplies Units Sold by profit-per-cookie per row, then sums. SUMX needed because row math must happen before aggregation.



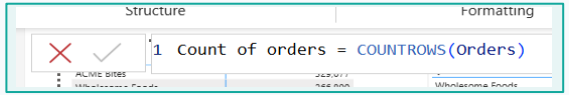
Measure

Count of Orders

700 total · e.g. ACME: 206

```
Count of orders = COUNTROWS(Orders)
```

Counts all rows in the Orders table. With no filter = 700. Filter context (e.g. a customer) narrows the count dynamically.



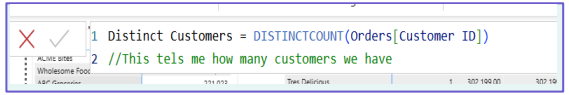
Measure

Distinct Customers

5 distinct customers

```
Distinct Customers = DISTINCTCOUNT(Orders[Customer ID])
//This tells me how many customers we have
```

Counts unique Customer ID values. DAX comment (//) used to annotate purpose. Returns 5 distinct customers.



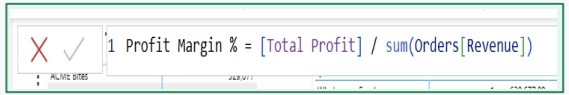
Measure

Profit Margin %

Overall: 57.93% · Choc Chip: 60.00%

```
Profit Margin % = [Total Profit] / sum(Orders[Revenue])
```

References another measure ([Total Profit]) divided by total revenue. Both change with filter context. Formatted as % in visuals.



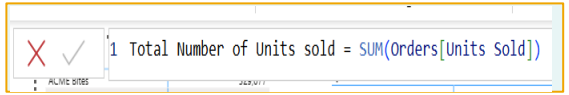
Measure

Total Units Sold

1,125,806 total units

```
Total Number of Units sold = SUM(Orders[Units Sold])
```

Simple SUM aggregation of units sold. Returns 1,125,806 total. Re-evaluates per customer/product slice automatically.



All DAX Formulas — Summary

Calculated Column

Profit

657 distinct values · Decimal number

```
Profit = Orders[Revenue] - Orders[Cost]
```

Computes net profit row-by-row. Stored in the model. Increases file size but enables row-level filtering by profit value.

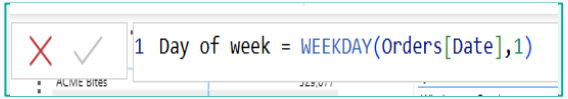
Calculated Column

Day of Week

7 distinct values (1–7) · Whole number

```
Day of week = WEEKDAY(Orders[Date], 1)
```

Extracts the day number (1=Sunday, 7=Saturday) from each order's date. Enables the bar chart visual showing orders by weekday.



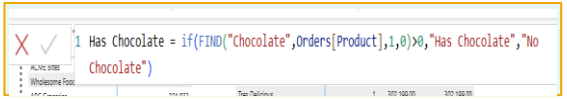
Calculated Column

Has Chocolate

Text: Has Chocolate / No Chocolate

```
Has Chocolate = IF(FIND("Chocolate",Orders[Product],1,0)>0,"Has Chocolate","No Chocolate")
```

Uses FIND( ) to detect "Chocolate" in the Product name. IF( ) converts the numeric FIND result to a readable text label.



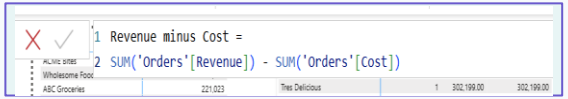
Quick Measure

Revenue minus Cost

Single aggregated value per context

```
Revenue minus Cost = SUM(Orders[Revenue]) - SUM(Orders[Cost])
```

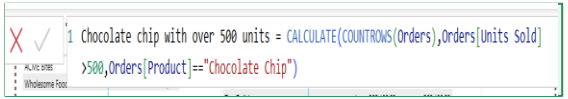
Auto-generated by Power BI's Quick Measure wizard. A Measure — aggregates dynamically. No storage overhead. Responds to filter context.



Measure (CALCULATE)

Choc Chip > 500 Units

185 orders matched

```
Chocolate chip with over 500 units = CALCULATE(COUNTROWS(Orders), Orders[Units Sold]>500, Orders[Product]=="Chocolate Chip")
```

CALCULATE( ) rewrites the filter context. Counts only rows where both filters pass simultaneously. Returns 185.